

INSTITUTO POLITÉCNICO DE BEJA

Escola Superior de Tecnologia e Gestão

Curso de Tecnologias Web e Dispositivos Móveis

Programação de Aplicação do Lado do Cliente · 2.º Semestre · 2025/2026

Cofrinho de Metas

Aplicação Android para gestão de poupanças pessoais por objetivo

Autor

Ronaldino Fernandes Chihita — n.º 27757

Docentes

Luís Garcia · Luís Rosário · Mihai Lupan

Beja, julho de 2026

Índice

1. Introdução.....	4
1.1 Enquadramento e regime individual.....	4
1.2 Estrutura do documento	4
2. Organização do trabalho e ferramentas.....	5
2.1 Ferramentas utilizadas	5
2.2 Organização em sprints.....	5
2.3 Estrutura do repositório	5
3. Análise do problema.....	6
3.1 O problema	6
3.2 Sistemas semelhantes analisados.....	6
Piggy Savings Tracker	6
Como o Cofrinho de Metas se diferencia.....	6
3.3 Persona — Sofia Marques (Poupadora Recorrente)	6
Motivações principais.....	7
Frustrações.....	7
3.4 Cenário de utilização — Distribuir a poupança semanal pelas metas	7
3.5 Análise hierárquica de tarefas (HTA).....	8
4. Funcionalidades da aplicação.....	9
4.1 Priorização para a demonstração.....	9
5. Desenho da interface da aplicação.....	10
5.1 Dispositivo alvo, orientação e estilos de interação.....	10
5.2 Sistema de design	10
5.3 Ecrãs implementados.....	10
Ecrã 1 — Lista de Metas	10
Ecrã 2 — Detalhe da Meta	11
Ecrã 3 — Criar / Editar Meta.....	12
5.4 Fluxo de navegação	13
5.5 Evolução do desenho — baixa para alta fidelidade	14
5.6 Princípios de usabilidade — Heurísticas de Nielsen.....	14
Visibilidade do estado do sistema	14
Correspondência com o mundo real	14
Prevenção de erros	15
Reconhecimento em vez de memorização	15
5.7 Sítio web	15

6. Desenho da base de dados.....	16
6.1 Modelo conceptual.....	16
6.2 Diagrama Entidade-Relação	16
7. Conceção da base de dados	17
7.1 Comandos de criação — SQL equivalente	17
7.2 Queries principais.....	17
8. Programação da lógica da aplicação.....	19
8.1 Stack tecnológica	19
8.2 Arquitetura MVVM — fluxo de dados unidirecional	19
8.3 Operações CRUD implementadas	19
8.4 Uso de Coroutines e Flow.....	19
9. Página web da aplicação e screencast.....	20
9.1 Sítio web de divulgação	20
9.2 Screencast	20
10. Conclusão	21
10.1 Outros aspetos desenvolvidos.....	21
10.2 Trabalho futuro	21
10.3 Reflexão pessoal.....	21
Anexo A — Contribuições e uso de Inteligência Artificial	23
A.1 Contribuições	23
A.2 Uso de ferramentas de IA	23
Anexo B — Repositório de código	24
Referências.....	25

1. Introdução

O presente relatório documenta o desenvolvimento do projeto Cofrinho de Metas, uma aplicação móvel Android para gestão de poupanças pessoais organizadas por objetivos ("metas"). O trabalho foi desenvolvido individualmente no âmbito da unidade curricular Programação de Aplicação do Lado do Cliente (PAC), do 2.º semestre do Curso de Tecnologias Web e Dispositivos Móveis da Escola Superior de Tecnologia e Gestão de Beja.

1.1 Enquadramento e regime individual

A equipa original inscrita na UC PAC foi constituída por Ronaldino Fernandes Chihita (n.º 27757) e Jorge Júnior (n.º 27562), a mesma equipa do projeto Opticart do 1.º semestre. Ao início do 2.º semestre surgiram divergências metodológicas quanto ao âmbito da implementação — nomeadamente quanto ao nível de profundidade técnica a assumir, às ferramentas a utilizar, e ao esforço a dedicar à componente de código face à componente de análise. Após discussão, a equipa acordou que cada elemento avançaria com um projeto próprio e autónomo, cada um com âmbito reduzido e defensável individualmente. O presente relatório reporta o trabalho desenvolvido nesse regime pelo autor.

O Cofrinho de Metas é, portanto, uma aplicação nova — deliberadamente pequena, com uma persona, um cenário, três ecrãs e duas entidades — de modo a poder ser inteiramente concebida, implementada, documentada e defendida por uma só pessoa.

1.2 Estrutura do documento

A aplicação foi implementada nativamente em Android com Kotlin, Jetpack Compose e Room, seguindo o padrão arquitetural MVVM. O idioma é totalmente português europeu (pt-PT). O código-fonte está publicado em <https://github.com/ronaldinochihita/CofrinhoDeMetas> e o sítio web de divulgação em <https://cofrinho-met.netlify.app/>.

O relatório aborda, pela seguinte ordem: organização e ferramentas (§2), análise do problema com persona, cenário e análise hierárquica de tarefas (§3), lista de funcionalidades (§4), desenho da interface e do sítio web (§5), modelo e conceção da base de dados com queries documentadas (§6 e §7), programação da lógica com arquitetura MVVM (§8), divulgação (§9) e conclusão (§10). Encerra-se com Anexos sobre contribuições, uso de IA, referências e ligação ao repositório.

2. Organização do trabalho e ferramentas

2.1 Ferramentas utilizadas

- IDE: Android Studio Ladybug (2024.2)
- Linguagem e build: Kotlin 2.0.21, Gradle 8.9, Android Gradle Plugin 8.7.2, KSP para Room
- Controlo de versões: Git + GitHub — repositório público em github.com/ronaldinochihita/CofrinhoDeMetas
- Prototipagem inicial: lápis e papel, formato A4
- Prototipagem em código: @Preview do Jetpack Compose (16 previews em 3 ecrãs)
- Hosting do sítio web: Netlify (deploy estático via Netlify Drop, HTTPS + CDN global)
- Documentação: Markdown para rascunhos, python-docx e python-pptx para entregáveis finais
- Comunicação com os docentes: aulas presenciais e Moodle da UC
- Assistente de IA: Claude (Anthropic) — ver Anexo A para descrição detalhada

2.2 Organização em sprints

O trabalho foi organizado em quatro sprints curtos, com um objetivo entregável por sprint:

- Sprint 1 — Análise: domínio, persona, cenário de utilização e análise hierárquica de tarefas
- Sprint 2 — Desenho: esboços de baixa fidelidade em papel, decisão da navegação, escolhas de estilo e paleta
- Sprint 3 — Implementação: três ecrãs em Jetpack Compose, integração com Room, ViewModel + StateFlow
- Sprint 4 — Documentação e entrega: relatório, apresentação, sítio web, screencast

2.3 Estrutura do repositório

CofrinhoDeMetas/	
├─ android-app/	Projeto Kotlin + Jetpack Compose (abre em Android Studio)
├─ website/	Sítio web estático (HTML + CSS)
├─ database/	schema.sql + queries.sql documentadas
├─ design/	Screenshots + diagrama de navegação (PNG)
├─ docs/	Relatório, apresentação, plano de testes, PDFs de layout
├─ screencast/	Guião do vídeo demo (~3 min)
├─ netlify.toml	Configuração de deploy (SEO + headers de segurança)
└─ README.md	Ponto de entrada

3. Análise do problema

3.1 O problema

Muitas pessoas guardam dinheiro sem um destino claro, o que reduz a motivação para poupar. Sem visibilidade sobre o progresso, torna-se difícil manter a disciplina — a poupança perde-se em gastos correntes ou é redirecionada para prioridades de curto prazo. Uma aplicação que permita atribuir cada euro a uma meta concreta e visualizar o progresso pode aumentar significativamente essa disciplina.

3.2 Sistemas semelhantes analisados

Para enquadrar o desenho do Cofrinho de Metas foi analisada uma aplicação existente no mesmo domínio — a poupança pessoal orientada por objetivos.

Piggy Savings Tracker

Piggy Savings Tracker (Vova Studio, iOS e Android, freemium com anúncios, +5 M downloads na Play Store) é uma das apps mais populares do segmento "goal-based savings". A metáfora central é o "mealheiro digital": o utilizador cria vários mealheiros temáticos e vê o dinheiro entrar em cada um deles ao longo do tempo.

Pontos positivos: visualização clara do progresso por meta com barras e percentagens; permite fotografia de contexto para cada meta; suporta várias moedas.

Pontos negativos: publicidade agressiva na versão gratuita (anúncios de ecrã inteiro); inexistência de tradução para português europeu — apenas português do Brasil; exige criação de conta e sincronização com serviços proprietários, dificultando o uso offline; não permite personalização de cor por meta; tipografia genérica.

Lacunas relevantes: ausência de uma variante 100 % offline sem publicidade; sem localização real para o mercado português; scannability fraca quando há várias metas ativas.

Como o Cofrinho de Metas se diferencia

O Cofrinho de Metas explora deliberadamente estas lacunas: interface totalmente em português europeu, funcionamento 100 % offline (base Room local, sem dependências externas), ausência total de publicidade, e personalização por meta em dois eixos independentes — cor e ícone — que se propagam para o ícone, a percentagem e a barra de progresso.

3.3 Persona — Sofia Marques (Poupadora Recorrente)

Contexto pessoal. Sofia Marques tem 28 anos, é técnica de contabilidade numa empresa agrícola no concelho de Beja, solteira, vive sozinha num T1 arrendado no Bairro da Conceição. Rendimento líquido mensal cerca de 1.100 €. Tem alguma poupança acumulada e boas relações com a família em Ferreira do Alentejo.

Perfil digital. Utiliza smartphone Samsung Galaxy A54 diariamente entre 3 e 4 horas. Usa banca online (Millennium BCP), MB WAY, Instagram e TikTok. Já experimentou o Notion e o Google Sheets para acompanhar poupanças mas abandonou ambos — o primeiro por ser complexo demais, o segundo por ser rígido demais.

Motivações principais

- Poupar cerca de 800 € para uma viagem à Toscana no verão, com a melhor amiga.
- Constituir um fundo de emergência equivalente a 3 meses de despesas fixas (1.500 €).
- Substituir o portátil pessoal, com 7 anos, por um modelo novo (cerca de 900 €).
- Manter várias metas em paralelo sem perder de vista o progresso de cada uma.
- Ver progresso visual — precisa da recompensa psicológica da barra a crescer.

Frustrações

- A folha de Excel ficou desatualizada ao fim de duas semanas.
- A app do banco separa os movimentos por data, não por propósito.
- Quando não vê progresso rapidamente, deixa de poupar.
- Odeia apps que exigem criação de conta antes de qualquer utilização.

"Se eu não vejo a barrinha a crescer, deixo de poupar. Preciso de olhar para o telemóvel ao domingo à noite e sentir que a Toscana está mais perto."

Credibilidade. Segundo o INE (2024), mais de 95 % da população portuguesa entre 25-34 anos utiliza Internet diariamente em dispositivos móveis, e cerca de 70 % já efetuou transações financeiras via smartphone.

3.4 Cenário de utilização — Distribuir a poupança semanal pelas metas

Persona: Sofia Marques. **Quando:** sábado, 11h00. **Onde:** café da Rua da Casa Branca, em Beja. **Dispositivo:** Samsung Galaxy A54.

A Sofia recebeu o ordenado na sexta-feira à tarde. Já verificou o extrato: as despesas fixas do mês foram debitadas, e sobraram-lhe 90 € livres para poupar. Decide começar o ritual das últimas três semanas: sábado de manhã, café e distribuição da poupança da semana pelas metas.

Abre o Cofrinho de Metas. Aparecem-lhe as três metas ativas com o progresso atual — a Viagem à Toscana a 45 %, o Fundo de Emergência a 22 %, e o Portátil Novo a 66 %. Como o Portátil está prestes a atingir os 75 %, a Sofia decide priorizar o Fundo, que anda muito atrás do plano ideal.

Toca no cartão do Fundo, consulta o detalhe, volta atrás, entra na Viagem, toca em "Adicionar depósito", introduz 30 €, escreve a nota "sábado semanal" e confirma. A barra sobe de 45 % para 49 %. Repete: 40 € para o Fundo, 20 € para o Portátil. No fim, olha para a lista com as três barras ligeiramente mais preenchidas e fecha a app.

3.5 Análise hierárquica de tarefas (HTA)

A HTA do cenário 3.4 decompõe o objetivo — distribuir a poupança semanal — em sub-tarefas, agrupadas por planos que explicitam a ordem de execução.

- 0. Distribuir a poupança semanal pelas metas
Plano 0: 1 → repetir 2 (por meta escolhida) → 3
 - 1. Aceder à lista de metas Plano 1: 1.1 → 1.2
 - 1.1 Abrir a aplicação
 - 1.2 Visualizar o progresso atual de cada meta
 - 2. Registrar um depósito numa meta Plano 2: repetitivo por meta
 - 2.1 Selecionar meta na lista (tocar no cartão)
 - 2.2 Tocar em "Adicionar depósito" (FAB)
 - 2.3 Inserir valor do depósito
 - 2.4 Inserir nota opcional
 - 2.5 Confirmar
 - 2.6 Ver a barra de progresso atualizada
 - 3. Terminar a sessão Plano 3: 3.1
 - 3.1 Voltar à lista e fechar a aplicação

A HTA demonstra dois padrões: plano principal sequencial-repetitivo (a tarefa 2 é repetida N vezes, uma por meta escolhida) e sub-plano 2 estritamente sequencial (cada campo tem de ser preenchido antes do seguinte). Isto justifica o formulário linear e a validação inline usados no diálogo de novo depósito. A tarefa 2.6 foi mantida no plano por ser o principal reforço motivacional da persona.

4. Funcionalidades da aplicação

A aplicação implementa nove funcionalidades, agrupadas em essenciais (as que servem diretamente o cenário 3.4) e complementares.

#	Funcionalidade	Descrição
F1	Listar metas	Cartões com barra de progresso, percentagem, valor/objetivo
F2	Criar meta	Nome, valor objetivo, prazo, cor e ícone
F3	Editar meta	Modificar qualquer campo — formulário reutilizado
F4	Eliminar meta	Remove meta e depósitos associados por CASCADE
F5	Pesquisar meta	Filtro por nome em tempo real
F6	Ver detalhe	Progresso ampliado + histórico de depósitos
F7	Adicionar depósito	Valor obrigatório + nota opcional (diálogo modal)
F8	Eliminar depósito	Ícone lixo por linha, remove individualmente
F9	Tema claro/escuro	Segue automaticamente a definição do sistema

4.1 Priorização para a demonstração

Na apresentação e no screencast, a demonstração ao vivo cobre o fluxo central pelas quatro funcionalidades essenciais (F1, F2, F6, F7) e, no fim, ilustra brevemente a eliminação (F4) para fechar o ciclo de vida da meta.

5. Desenho da interface da aplicação

5.1 Dispositivo alvo, orientação e estilos de interação

O dispositivo alvo é o smartphone Android. O sistema alvo é Android 8.0 (API 26) como mínimo — cobre mais de 95 % do parque de dispositivos ativos em Portugal — com API 35 como alvo de compilação.

A aplicação é desenhada para uso em orientação portrait, natural para leitura de lista e para o toque com o polegar. Estilos usados: tap (ação primária) e scroll vertical. Não são usados long-press nem swipe — nenhuma ação está escondida atrás de gestos "descobríveis". Em particular, foi rejeitada a hipótese swipe-para-eliminar para prevenir eliminações acidentais.

5.2 Sistema de design

A aplicação assenta em Material Design 3, com uma paleta cromática dominada pelo verde-escuro (#2E7D32) como cor primária. As cores individuais de cada meta são escolhidas pelo utilizador a partir de uma paleta de oito cores calibradas para contraste WCAG AA.

A tipografia segue as escalas do Material 3: bodyLarge (16 sp) para corpo, titleMedium (18 sp bold) para nomes de meta, titleLarge (22 sp bold) para destaques. A percentagem de progresso nos cartões da Lista usa titleMedium a bold, na cor da meta, para leitura imediata mesmo em relance.

5.3 Ecrãs implementados

Ecrã 1 — Lista de Metas

Este é o ponto de entrada da aplicação. A barra de topo apresenta o título centrado. Um campo de pesquisa filtra a lista em tempo real. Cada meta é apresentada num cartão com o ícone circular à esquerda (a cor da meta), o nome, o par "valor depositado / valor objetivo", a percentagem em bold à direita e uma barra de progresso. O FAB estendido "Nova meta" dá acesso ao ecrã de criação.

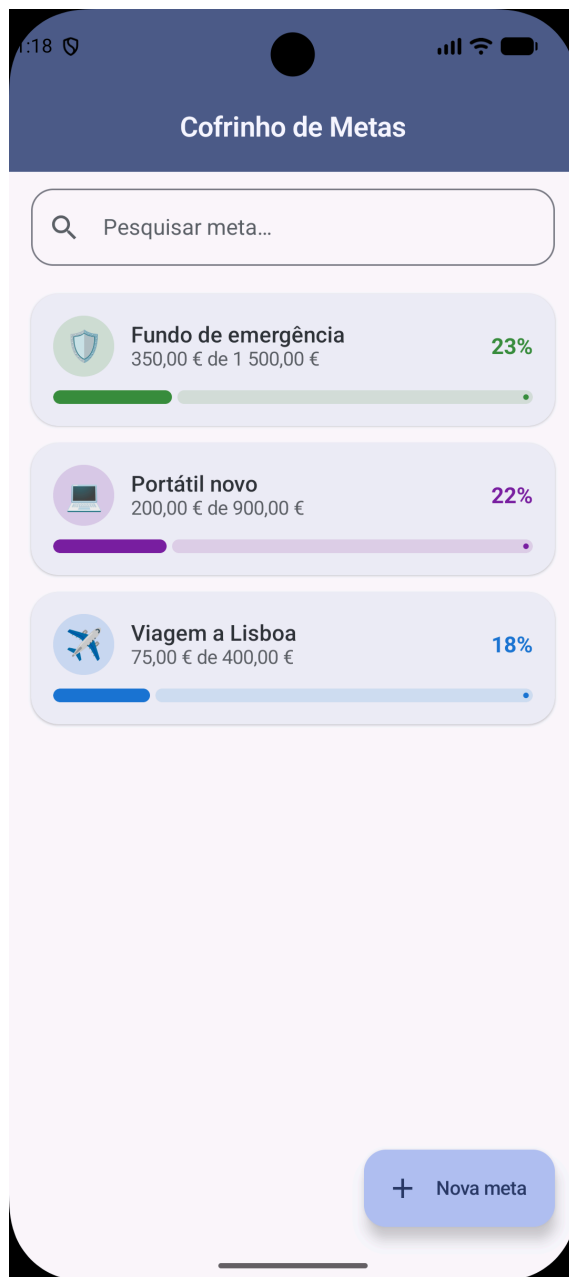


Figura 1 — Ecrã Lista de Metas com três metas ativas

Ecrã 2 — Detalhe da Meta

Ao tocar num cartão da lista, o utilizador entra neste ecrã. A barra de topo mostra o nome da meta. O cartão de resumo apresenta o ícone, o objetivo, uma barra de progresso maior e as duas grandezas "Poupado" e "Falta" lado a lado. A secção "Depósitos" lista todos os depósitos por ordem cronológica inversa. O FAB "Adicionar depósito" abre o diálogo modal.

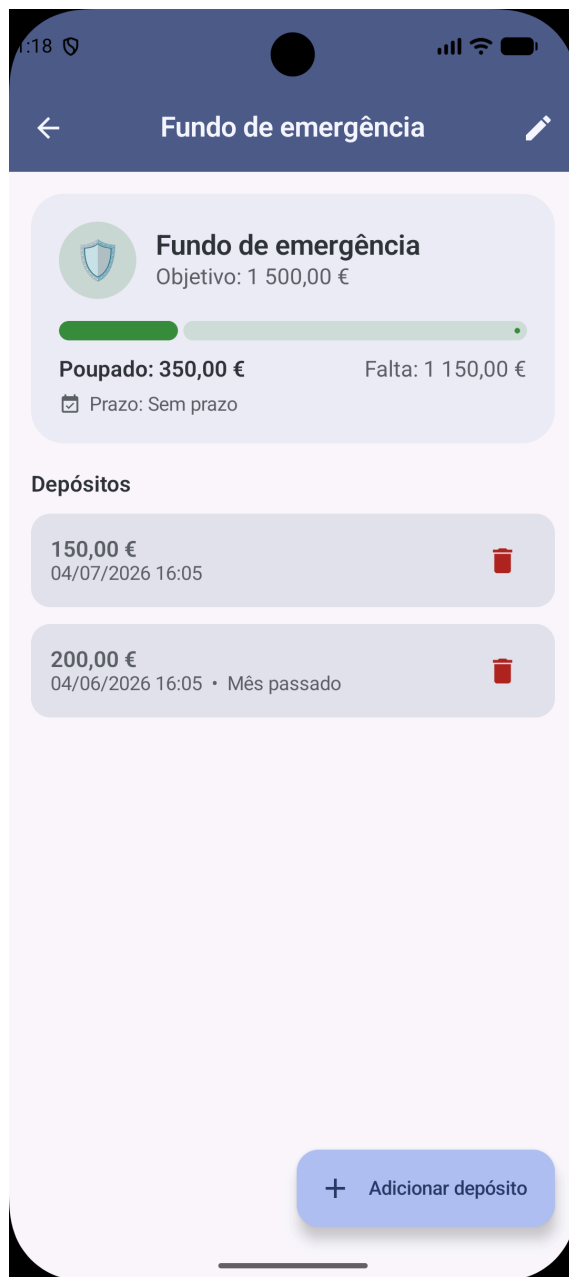
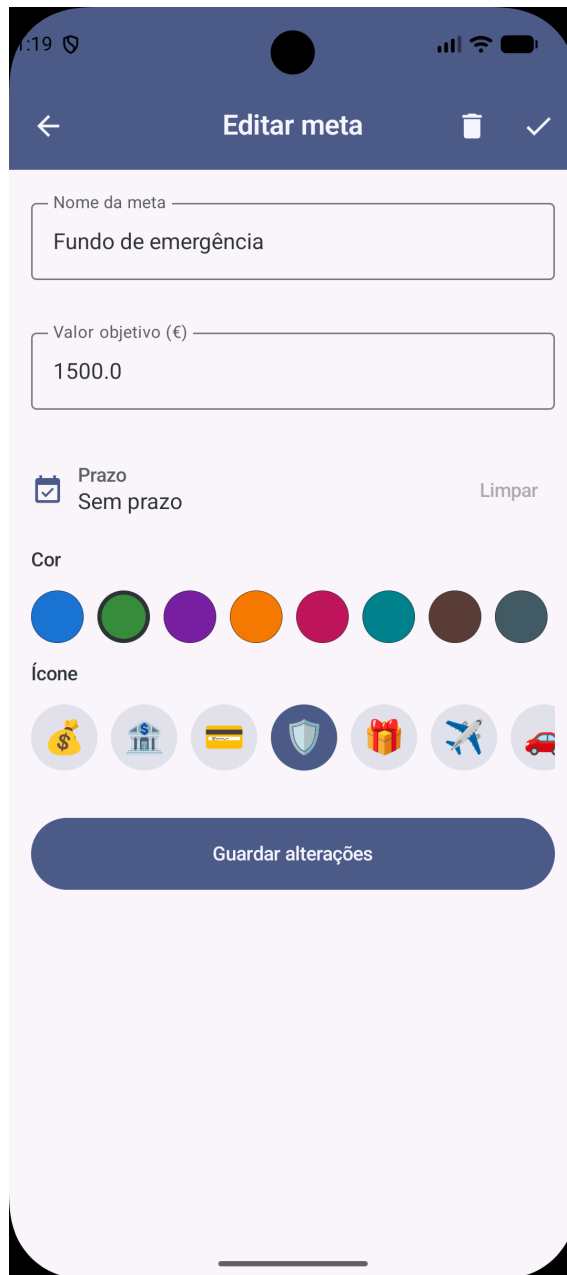


Figura 2 — Ecrã Detalhe da meta "Fundo de emergência"

Ecrã 3 — Criar / Editar Meta

Este ecrã é reutilizado para criação e edição. O título muda conforme o modo. Em modo edição, expõe adicionalmente o ícone de caixote de lixo (com AlertDialog de confirmação). O formulário contém os campos Nome, Valor objetivo, Prazo, Cor e Ícone (fila horizontal de 18 emojis). O botão principal "Guardar alterações" no fundo é largo e proeminente.



19

Editar meta

Nome da meta

Fundo de emergência

Valor objetivo (€)

1500.0

Prazo

☒ Sem prazo

Limpar

Cor

Ícone

Guardar alterações

Figura 3 — Ecrã Editar Meta

5.4 Fluxo de navegação

A aplicação está estruturada em três ecrãs (Lista, Detalhe, Editar/Criar) e um diálogo modal (Novo Depósito), com navegação em profundidade a partir da Lista de Metas. Todos os ecrãs oferecem um caminho curto de regresso (seta "voltar" na barra de topo), cumprindo o princípio de "controlo do utilizador".

DIAGRAMA DE NAVEGAÇÃO

Gestão de metas — ponto de entrada, detalhe, depósitos e edição

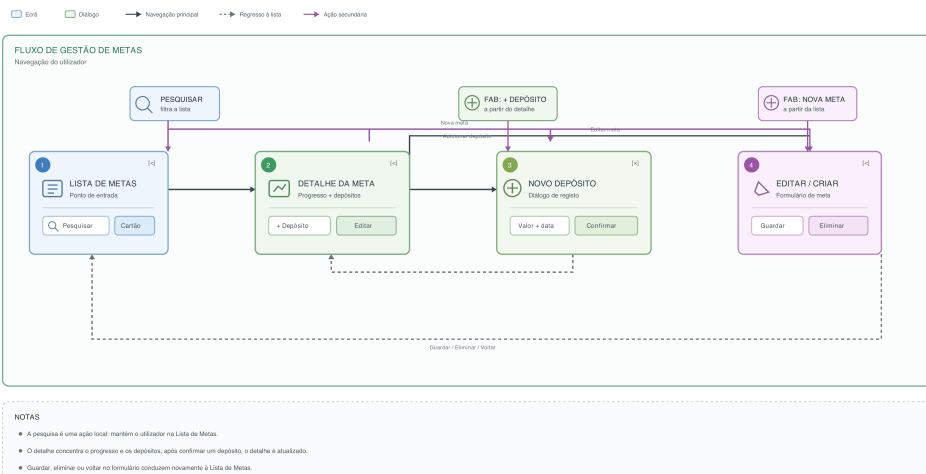


Figura 4 — Diagrama de navegação da aplicação

5.5 Evolução do desenho — baixa para alta fidelidade

Face aos esboços iniciais em papel, foram tomadas várias decisões durante a implementação:

- Introdução de cor por meta — os esboços eram monocromáticos.
- Percentagem numérica visível na lista — originalmente só existia a barra.
- Resumo bipartido "Poupado / Falta" no detalhe — reduz o esforço cognitivo.
- Eliminação escondida atrás do ecrã de edição — previne eliminações acidentais.
- Duplo caminho de gravação no formulário — ícone ✓ e botão principal.
- Estado vazio ilustrado e informativo — mensagem clara com próxima ação sugerida.

5.6 Princípios de usabilidade — Heurísticas de Nielsen

As decisões de desenho foram sustentadas nas Heurísticas de Nielsen. Destacam-se as quatro com evidência mais concreta no protótipo:

Visibilidade do estado do sistema

Em cada cartão da Lista, o utilizador vê três indicadores redundantes do estado (valor/objetivo em texto, percentagem em bold a cor da meta, barra de progresso). No Detalhe, o estado é reforçado com a bipartição "Poupado / Falta".

Correspondência com o mundo real

Vocabulário PT-PT (meta, poupado, falta, depósito). Moeda formatada em Locale pt-PT ("1 500,00 €"), datas em dd/MM/yyyy HH:mm. Zero terminologia técnica exposta na UI.

Prevenção de erros

Validação inline (valor > 0), erro em vermelho sob o campo (nunca em Toast). Botão Guardar desativado enquanto houver erros. Eliminar meta exige AlertDialog de confirmação, e está escondido atrás de "entrar em edição".

Reconhecimento em vez de memorização

Cor + ícone por meta propagados a Lista/Detalhe/resumo. Estado vazio da Lista dá a próxima ação sugerida em vez de deixar o utilizador perante um ecrã em branco.

5.7 Sítio web

Página única, estática, responsiva (mobile-first), com secções Hero + mockup, Funcionalidades, Screencast, Tecnologias e Autor. Publicada em <https://cofrinho-met.netlify.app/> (Netlify, HTTPS + CDN global). Configuração de SEO, Open Graph e cabeçalhos de segurança em netlify.toml.

6. Desenho da base de dados

6.1 Modelo conceptual

O domínio da aplicação é modelado por duas entidades — deliberadamente simples — com uma relação um-para-muitos entre elas:

- Meta (id, nome, valorObjetivo, dataLimite?, corHex, iconeChave, dataCriacao)
- Depósito (id, metald FK, valor, data, nota)

6.2 Diagrama Entidade-Relação

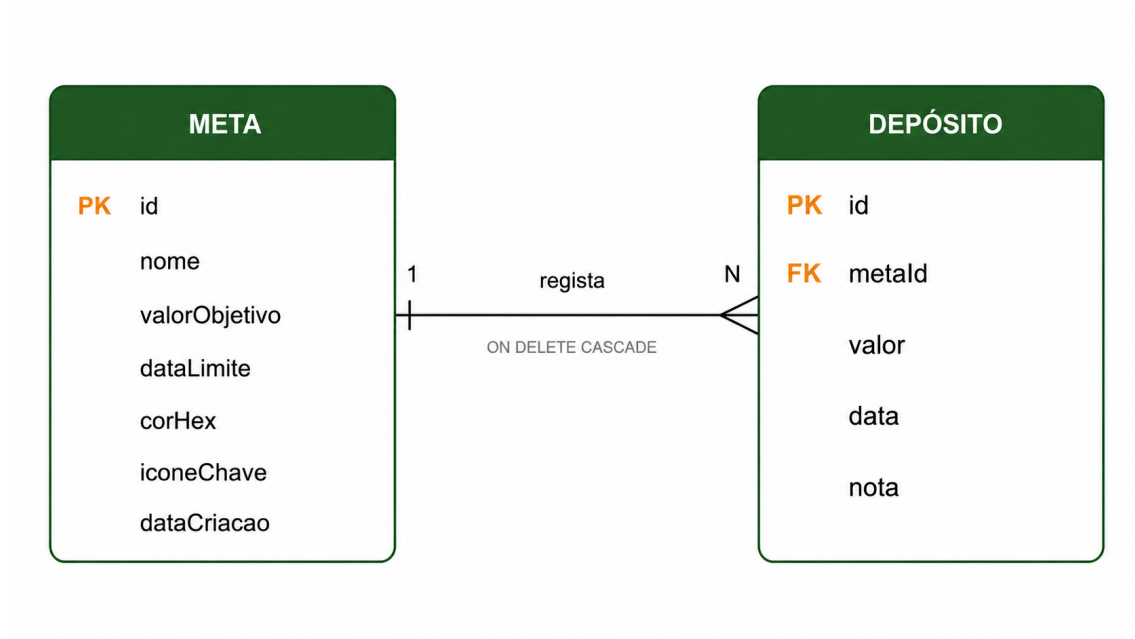


Figura 5 — Diagrama Entidade-Relação da base de dados (notação Crow's foot)

A eliminação de uma Meta despoleta a eliminação dos seus Depósitos por regra ON DELETE CASCADE. Um objeto auxiliar MetaComTotal (query LEFT JOIN + SUM) devolve cada Meta já enriquecida com o total depositado e a contagem de depósitos, evitando cálculos na camada de UI e mantendo a lista completamente reativa.

7. Conceção da base de dados

A base de dados foi implementada com Room 2.6 sobre SQLite. O Room é a biblioteca oficial da Google para persistência em Android — assenta sobre SQLite, valida queries em tempo de compilação (@Query) e integra-se com Kotlin Coroutines e Flow para reatividade.

7.1 Comandos de criação — SQL equivalente

```
CREATE TABLE meta (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    nome          TEXT      NOT NULL,
    valorObjetivo REAL      NOT NULL,
    dataLimite    INTEGER,
    corHex        TEXT      NOT NULL DEFAULT '#4CAF50',
    iconeChave    TEXT      NOT NULL DEFAULT 'savings',
    dataCriacao   INTEGER NOT NULL
);

CREATE TABLE deposito (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    metaId        INTEGER NOT NULL,
    valor         REAL     NOT NULL,
    data          INTEGER NOT NULL,
    nota          TEXT      NOT NULL DEFAULT '',
    FOREIGN KEY (metaId) REFERENCES meta(id) ON DELETE CASCADE
);

CREATE INDEX idx_deposito_metaId ON deposito(metaId);
```

7.2 Queries principais

Q1 — Listar metas com total poupado e número de depósitos

```
SELECT m.*,
       IFNULL(SUM(d.valor), 0.0) AS totalDepositado,
       COUNT(d.id)              AS numDepositos
FROM meta m
LEFT JOIN deposito d ON d.metaId = m.id
GROUP BY m.id
ORDER BY m.dataCriacao DESC;
```

Usa LEFT JOIN para incluir metas sem depósitos e IFNULL para tratar o NULL do agregado. Devolvida como Flow<List<MetaComTotal>> — a UI atualiza-se sozinha quando novos depósitos são inseridos.

Q2 — Obter uma meta específica com o seu total

```
SELECT m.*, IFNULL(SUM(d.valor), 0.0) AS totalDepositado, COUNT(d.id) AS numDepositos
FROM meta m
LEFT JOIN deposito d ON d.metaId = m.id
WHERE m.id = :id
GROUP BY m.id;
```

Q3 — Pesquisa por nome (LIKE parcial)

```
SELECT m.*, IFNULL(SUM(d.valor), 0.0) AS totalDepositado, COUNT(d.id) AS numDepositos
FROM meta m
LEFT JOIN deposito d ON d.metaId = m.id
WHERE m.nome LIKE '%' || :termo || '%'
GROUP BY m.id
ORDER BY m.dataCriacao DESC;
```

Q4 — Total global poupado em todas as metas

```
SELECT IFNULL(SUM(valor), 0.0) FROM deposito;
```

Q5 — Depósitos de uma meta (mais recentes primeiro)

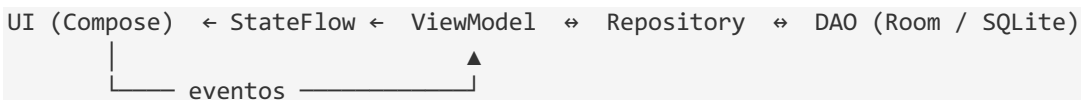
```
SELECT * FROM deposito
WHERE metaId = :metaId
ORDER BY data DESC;
```

8. Programação da lógica da aplicação

8.1 Stack tecnológica

Camada	Tecnologia	Justificação
Linguagem	Kotlin 2.0.21	Oficial Android, null-safety, sintaxe concisa
UI	Jetpack Compose (BOM 2024.10)	Framework declarativo, previews interativas
Design System	Material 3	Modo claro/escuro nativo, componentes acessíveis
Persistência	Room 2.6.1 (SQLite)	ORM oficial, queries reativas com Flow
Assincronia	Coroutines + Flow	Reatividade sobre a base — UI atualiza-se sozinha
Navegação	Navigation Compose 2.8	Rotas tipadas em CofrinhoNavHost.kt
Arquitetura	MVVM	Um ViewModel por ecrã, StateFlow observável

8.2 Arquitetura MVVM — fluxo de dados unidirecional



A UI é puramente declarativa: recebe o estado (via `collectAsStateWithLifecycle()`) e emite eventos (funções lambda). O ViewModel prepara e expõe o estado através de `StateFlow`, e delega as operações de dados ao Repository. O Repository é a fachada única sobre os DAOs. Room gera o código dos DAOs em tempo de compilação a partir das interfaces anotadas com `@Dao`.

8.3 Operações CRUD implementadas

- Create: `MetaDao.inserir()` e `DepositoDao.inserir()` — anotadas com `@Insert`.
- Read: `observarComTotais()`, `observarPorMeta()`, `pesquisar()` — anotadas com `@Query`, devolvem `Flow<T>` para reatividade.
- Update: `MetaDao.atualizar()` — anotada com `@Update`.
- Delete: `MetaDao.eliminar()` e `DepositoDao.eliminar()` — anotadas com `@Delete`. Cascade automático nos depósitos ao eliminar meta.

8.4 Uso de Coroutines e Flow

Queries de leitura devolvem `Flow<T>`: a UI subscreve e recompõe-se automaticamente quando os dados mudam. Operações de escrita são `suspend` — executam num dispatcher secundário (`Dispatchers.IO` gerido pelo Room), libertando a thread principal.

No ViewModel, os fluxos são transformados com operadores como `flatMapLatest` (por exemplo, para reagir à pesquisa: cada carácter digitado cancela a query anterior e emite uma nova). O ciclo de vida é gerido por `viewModelScope`.

9. Página web da aplicação e screencast

9.1 Sítio web de divulgação

URL em produção: <https://cofrinho-met.netlify.app/>

Página estática single-page, responsiva, com as secções: Hero (título + mockup do telemóvel), Funcionalidades (grid de 6 cartões), Screencast, Tecnologias e Autor. Cores alinhadas com a identidade da app (verde primário + dourado como acento).

Publicada no Netlify (hosting estático gratuito, HTTPS + CDN global). O deploy foi feito via Netlify Drop, arrastando a pasta `website/` para `app.netlify.com/drop`. O sítio inclui SEO com Open Graph tags para preview em redes sociais, favicon SVG inline com o símbolo €, e cabeçalhos HTTP de segurança e cache aplicados via configuração do host.

9.2 Screencast

O screencast está gravado e disponível em `screencast/screencast.webm` no repositório, e reproduzível directamente na secção "Screencast" do sítio web em produção. Foi gravado a partir de emulador Android (Pixel 9a, API 37) com narração em português europeu. Cobre os principais fluxos da aplicação:

- Ecrã inicial — Lista de metas com progresso visual e pesquisa em tempo real.
- Criação de nova meta — formulário com nome, valor, prazo, cor e ícone.
- Detalhe da meta e adição de depósitos com atualização imediata da barra.
- Edição e eliminação de metas (com diálogo de confirmação).
- Fecho com identidade visual e menção à arquitetura MVVM.

10. Conclusão

O protótipo funcional Cofrinho de Metas foi desenvolvido em regime individual, com foco deliberado num âmbito reduzido para permitir profundidade de execução. Foram entregues:

- Aplicação Android nativa em Kotlin com Jetpack Compose e Room, funcional em emulador e dispositivo real (mínimo Android 8 / API 26).
- Três ecrãs implementados (Lista, Detalhe, Editar/Criar) que cobrem integralmente o cenário de utilização estudado (secção 3.4).
- Nove funcionalidades operacionais (F1 a F9), organizadas em essenciais e complementares.
- Base de dados Room com esquema SQL documentado (secção 7.1) e cinco queries principais explicadas em detalhe (secção 7.2).
- Arquitetura MVVM com StateFlow, Repository e DAOs — código organizado em camadas.
- Sítio web de divulgação publicado em URL público com HTTPS.
- Persona, cenário, análise hierárquica de tarefas e quatro heurísticas de Nielsen com evidência concreta no protótipo.

10.1 Outros aspetos desenvolvidos

Para além do exigido pelo enunciado, foram desenvolvidos os seguintes aspetos adicionais, valorizados na avaliação:

- Deploy do sítio web em Netlify com HTTPS e Open Graph tags para partilha social.
- 16 previews (@Preview) do Jetpack Compose para acelerar iteração visual sem emulador.
- Ícone launcher redesenhado com adaptive icon completo (foreground, background e monochrome para themed icons do Android 13+).
- Seletor de ícones baseado em 18 emojis Unicode organizados por 6 categorias.
- Coerência total com o TG2 da unidade curricular Sistemas Interativos (mesma persona, cenário, HTA e heurísticas de Nielsen).

10.2 Trabalho futuro

As iterações futuras poderão endereçar: (i) um ecrã de estatísticas globais (evolução do total poupado ao longo do tempo, ritmo médio); (ii) exportação de dados para CSV; (iii) notificações locais como reforço motivacional; (iv) sincronização entre dispositivos, o que exigiria backend. Nenhuma destas evoluções altera o núcleo do modelo de dados nem a arquitetura MVVM adotada.

10.3 Reflexão pessoal

O projeto serviu como consolidação prática dos conceitos apresentados na UC PAC — em particular, do padrão MVVM aplicado a Compose, do uso de StateFlow para reatividade da UI, e da importância de manter o âmbito controlado quando se trabalha sozinho. Se recomeçasse, começaria mais cedo pelas @Preview do Compose, poupando tempo de iteração visual sem precisar de correr o emulador a cada mudança de UI.

Anexo A — Contribuições e uso de Inteligência Artificial

A.1 Contribuições

O projeto foi integralmente concebido e implementado por Ronaldino Fernandes Chihita (n.º 27757), em regime individual (ver secção 1.1). Inclui: desenho da persona e do cenário, análise hierárquica de tarefas, decisões de sistema de design e paleta cromática, implementação em Kotlin/Jetpack Compose de todos os ecrãs e ViewModels, modelo Room com as duas entidades e DAOs, navegação, integração de todos os componentes num protótipo funcional em dispositivo real, deploy do sítio web em Netlify, e redação dos entregáveis.

A.2 Uso de ferramentas de IA

Foi utilizado o assistente Claude (Anthropic) nos seguintes contextos:

- Discussão e refinamento da persona e do cenário de utilização.
- Revisão e formalização da análise hierárquica de tarefas.
- Discussão da estrutura do relatório e da apresentação.
- Revisão linguística em português europeu.
- Apoio na estruturação dos scripts geradores dos ficheiros Word e PowerPoint entregues.
- Sugestões para redesenho do ícone launcher (moeda com € em vetor XML).

Em momento algum a IA produziu trabalho que não tenha sido revisto, validado e ajustado pelo autor. Todas as decisões de desenho refletem o entendimento e a posição assumida pelo autor. O código Kotlin/Compose e Room da aplicação foi integralmente desenvolvido em Android Studio.

Anexo B — Repositório de código

GitHub: <https://github.com/ronaldinochihita/CofrinhoDeMetas>

Estrutura relevante do repositório:

- `android-app/` — projeto Kotlin + Jetpack Compose, abre em Android Studio Ladybug (2024.2).
- `android-app/app/src/main/java/pt/ipbeja/cofrinho/` — código-fonte organizado em `data/` (entidades, DAOs, repositório, seed) e `ui/` (ecrãs, ViewModels, tema, navegação).
- `database/` — script SQL de criação (`schema.sql`) e queries documentadas (`queries.sql`).
- `design/` — screenshots dos ecrãs e diagrama de navegação (PNG).
- `docs/` — relatório, guiões, PDFs de layout e wireframes.
- `website/` — sítio web de divulgação (HTML + CSS), publicado em Netlify.
- `screencast/` — guião do vídeo demo (~3 min).

A base de dados é local (Room sobre SQLite) e não requer credenciais nem serviços externos. Para executar: abrir a pasta `android-app/` no Android Studio, sincronizar o Gradle, e correr num emulador ou dispositivo físico (mínimo Android 8 / API 26).

Referências

- Nielsen, J. (1993). Usability Engineering. Academic Press.
- Norman, D. (2013). The Design of Everyday Things (Revised and Expanded Edition). Basic Books.
- Instituto Nacional de Estatística (2024). Sociedade da Informação e do Conhecimento — Inquérito à Utilização de Tecnologias da Informação e da Comunicação pelas Famílias.
- Google (2024). Material Design 3 Guidelines. <https://m3.material.io/>
- Google (2024). Android Developer Documentation — Jetpack Compose. <https://developer.android.com/jetpack/compose>
- Google (2024). Room Persistence Library. <https://developer.android.com/training/data-storage/room>